

Sebastian Revelo

Daniel Acosta

Mateo Gualdron

- a) ¿Qué características de los layouts explican las diferencias observadas entre ascenso de colina, recocido simulado y algoritmo genético?

Las diferencias se explican por la topología del espacio de búsqueda inducida por la relación de vecindad (mover un solo módulo a la vez). En `local_peak`, ascenso de colina obtuvo desviación 10.28 entre corridas (mejor=96, promedio=81), la presencia de máximos locales aislados hace que, dependiendo de la configuración inicial, el algoritmo quede atrapado en picos de distinta calidad, sin ninguna forma de escapar porque exige una mejora estricta. En `plateau_grid`, el promedio de ascenso de colina cae aún más (66.8, desviación 11.55): las mesetas son regiones con varios vecinos de puntaje igual o casi igual que violan directamente la condición de parada del algoritmo, deteniéndolo en cuanto no encuentra una mejora estrictamente mayor, incluso si existen mejores configuraciones "al otro lado" de la meseta. En `balanced_city`, un espacio de búsqueda mucho más grande con varias cuencas, la desviación de ascenso de colina se dispara a 30.99, mientras más grande y multimodal es el espacio, más determinante es dónde arranca el algoritmo. En contraste, recocido simulado y el algoritmo genético mantienen desviaciones bajas en los tres layouts (máximo 6.37 y 2.93 respectivamente), porque ambos incorporan mecanismos explícitos para escapar de óptimos locales: SA acepta movimientos peores con cierta probabilidad, y el genético mantiene diversidad en una población completa en vez de una sola trayectoria.

- b) ¿Qué efecto tuvieron la inicialización, la aleatoriedad y los parámetros principales sobre los resultados de los algoritmos de optimización?

La inicialización aleatoria es el factor dominante en ascenso de colina, como muestra su alta desviación en los tres layouts frente a SA y GA. Sobre los parámetros principales, en el algoritmo genético el tamaño de población tuvo un efecto claro: en `balanced_city`, `population=10` produjo desviación 2.93 y promedio 279.20, mientras que `population=80` bajó la desviación a 0.98 y subió el promedio a 281.80. Una población más grande da más diversidad y por tanto resultados más consistentes entre corridas, a costa de 8× más

evaluaciones (1010 vs 8080) y un tiempo casi 9 veces mayor. La probabilidad de mutación, en cambio, tuvo un efecto mucho más marginal en este mismo layout: variar de 0.01 a 0.3 no cambió el promedio (281.80 en ambos) y solo movió ligeramente la desviación (1.60 vs 0.98), sugiriendo que con población=40 y 100 generaciones, el cruce y la selección por torneo ya aportan suficiente diversidad, y la mutación no es el cuello de botella en este caso. Esto indica que no todos los parámetros pesan igual: la población fue más determinante que la mutación para la consistencia del resultado en este layout particular.

- c) ¿Qué compromiso observa entre calidad de la solución y costo computacional en los algoritmos de optimización?

El compromiso es claro y cuantificable: en `balanced_city`, ascenso de colina usa solo 60 evaluaciones y logra promedio 222.2, mientras que recocido simulado usa más o menos 758 evaluaciones (12.6× más) para llegar a 282.6, y el genético usa 4040 evaluaciones (67× más que ascenso de colina) para 281.6. El salto más grande en calidad ocurre entre ascenso de colina y los otros dos algoritmos (+60 puntos aprox.), mientras que entre SA y GA la diferencia de calidad es mínima pese a que GA usa alrededor de 5× más evaluaciones que SA. Esto sugiere que, en este layout, SA logra un mejor punto de equilibrio costo/calidad que el genético con los parámetros probados. El caso de población en GA refuerza este punto: duplicar población de 40 a 80 (8080 evaluaciones) solo ganó 2.6 puntos de promedio sobre población=10 (1010 evaluaciones). Un retorno marginal decreciente frente al costo computacional adicional.

- d) Identifique un caso en el que cambiar la profundidad modifique la decisión o el resultado del defensor. Explique por qué ocurre y relacione su respuesta con la función de evaluación y el efecto horizonte.

En `forked_routes`, con profundidad 2 el defensor elige la acción East y logra activar todas las terminales (victoria, 486 nodos explorados). Con profundidad 4, el defensor elige North en su lugar, y la partida termina en límite de rondas sin ganar (18499 nodos). El cambio de decisión ocurre porque, según las reglas del proyecto, el intruso simulado también busca con la misma profundidad que el experimento así que subir la profundidad no solo mejora la previsión del defensor, sino que también vuelve al intruso un oponente más efectivo. Es decir, la ruta East parecía óptima a 2 plies porque a esa distancia

la heurística no anticipaba que el intruso, buscando con más profundidad, podría interceptar o bloquear esa ruta de forma más eficaz. Esto es una variante del efecto horizonte: la búsqueda limitada no "ve" el peligro que aparece justo después del límite de profundidad, pero en este caso el peligro adicional no es solo estructural (una trampa en el mapa), sino que surge de que el propio adversario razona con mayor alcance.

- e) Explique por qué alfa-beta debe producir la misma decisión que Minimax bajo las mismas condiciones. Analice cómo la profundidad afecta el costo de la búsqueda y cómo el orden de las acciones afecta la eficacia de la poda alfa-beta.

Alfa-beta produce siempre la misma decisión que Minimax porque solo poda ramas que un jugador "normal" nunca elegiría explorar completamente: cuando el valor de un nodo ya garantiza que MIN (o MAX) nunca permitiría llegar a él porque existe una alternativa mejor más arriba en el árbol, seguir explorando esa rama no puede cambiar el valor final propagado a la raíz. La poda elimina trabajo que se repite, no información relevante para la decisión. Esto se confirmó en cada par comparado: en `single_terminal` (profundidad 2), ambos agentes eligen East y ganan, pero Minimax explora 112 nodos frente a 80 de alfa-beta; en `medium_defense`, ambos eligen North (872 vs 686 nodos); en `deceptive_distance`, ambos eligen North (668 vs 530 nodos); y en `tiny_defense` a profundidad 4, la reducción es más marcada aún (9657 vs 4138 nodos). En todos los casos la acción y el resultado final son idénticos, solo cambia el costo. Sobre la profundidad: el costo de Minimax crece exponencialmente con ella ($\text{ramificación}^{\text{profundidad}}$), como se ve en `tiny_defense` al pasar de 980 nodos en profundidad 2 a 9657 en profundidad 4 ($10\times$). Sobre el orden de las acciones: alfa-beta poda más eficazmente cuando las mejores acciones se exploran primero, porque eso permite que alpha y beta se ajusten rápido y corten las ramas restantes antes; con un orden desfavorable (peor acción primero), la poda es mínima y alfa-beta se acerca al costo de Minimax puro.

Tablas de pruebas

Layout	Algoritmo	Mejor	Promedio	Desviación	Evaluaciones	Tiempo
local Peak	Hill Climbing	96	81	10.28	15	0,000215
local Peak	simulatedAnnealing	96	90,8	6,37	755,6	0,013
local Peak	Genetic	96	96	0	4040	0,07
plateau_grid	Hill Climbing	80	66,8	11,55	15	0,0002
plateau_grid	simulatedAnnealing	94	94	0	755,8	0,0151
plateau_grid	Genetic	94	93,6	0,8	4040	0,081
balanced City	Hill Climbing	275	222,2	30,99	60	0,002
balanced City	simulatedAnnealing	283	282,6	0,8	758,4	0,035
balanced City	Genetic	283	281,6	1,2	4040	0,13

layout	Agente	Profundidad	Accion I	Resultado	N explorados	N Intruso
tiny_defense	MINIMAX	2	North	limite	980	854
tiny_defense	AlphaBeta	2	North	limite	855	854
tiny_defense	MINIMAX	4	Noorth	limite	9657	4390
tiny_defense	AlphaBeta	4	North	limite	4138	4390
single_terminal	MINIMAX	2	East	Victoria	112	73
single_terminal	AlphaBeta	2	East	Victoria	80	73
medium_defense	MINIMAX	2	North	limite	872	682
medium_defense	AlphaBeta	2	North	limite	686	682
deceptive	MINIMAX	2	North	limite	668	548
deceptive	AlphaBeta	2	North	limite	530	548
forked_routes	MINIMAX	2	East	Victoria	486	420
forked_routes	MINIMAX	4	North	Limite	18499	10008

Pruebas extra para ver el cambio de mutaciones y population
geneticAlgorithm | mejor=283.00 | promedio=281.80 | desviación=0.98 |
evaluaciones_promedio=4040.0 | tiempo_promedio=0.195805s

geneticAlgorithm | mejor=283.00 | promedio=281.80 | desviación=1.60 |
evaluaciones_promedio=4040.0 | tiempo_promedio=0.179831s

Prueba en forked

MinimaxAgent | profundidad=4 plies | acción_inicial=North | resultado=límite |
finalización=límite de rondas | nodos=18499 | nodos_intruso=10008 |
tiempo=0.285781s

MinimaxAgent | profundidad=2 plies | acción_inicial=East | resultado=victoria |
finalización=activación completa | nodos=486 | nodos_intruso=420 |
tiempo=0.021429s

Escriba una reflexión concisa sobre el uso de IA durante el desarrollo de la solución final, de acuerdo con la política indicada más adelante. Explique qué aprendió al revisar sus sugerencias, si los cambios fueron correcciones fundamentales o mejoras secundarias, qué tan difícil fue formular prompts útiles y si podría explicar la solución final en una sustentación. Si no realizó ninguna iteración con IA, indique por qué no le pareció necesario ni útil. No hay respuestas correctas o incorrectas; este punto busca que reflexione sobre el uso de IA en tareas de programación.

Uso de la IA

En este laboratorio se usó la IA para entender de mejor manera la estructura base del código, para entender como funcionaban los diferentes layouts, como cada parámetro afectaba en cada método de búsqueda o en los de adversarios. Además, para completar un mejor análisis se le pidió que diera algunas combinaciones de layouts y parámetros para obtener mejores datos y poder dar una mejor conclusión de como funciona cada punto del taller.